

课程作业 4：基于 PPO 算法的倒立摆控制

尧祥临 202518023406038 前沿交叉科学学院

1 任务介绍

本次作业的目标是实现近端策略优化（Proximal Policy Optimization, PPO）算法，并使用该算法在 OpenAI Gym 的 `CartPole-v1` 环境中训练智能体，实现对倒立摆的有效控制，目标是使智能体在尽可能多的时间步（500 步）内保持倒立摆的平衡。

2 模型架构

PPO 算法是一种基于策略梯度的强化学习方法，针对 TRPO 算法存在的重要性比例带来的大方差以及求解约束优化问题的困难等不足，在其基础上进行简化和改进，保留了策略稳定性。与课程 ppt 中介绍的 PPO 算法一致，这里采用了 Clip 截断式优化目标、GAE 广义优势估计和自适应 KL 惩罚的方法来提升训练的稳定性和效率。

2.1 Actor-Critic 框架

与使用蒙特卡洛采样直接估计回报的策略梯度方法不同，PPO 算法采用了 Actor-Critic 框架。其中 Actor 网络负责输出动作的概率分布，Critic 网络则负责估计状态值函数；在训练过程中，Critic 要学会准确估计当前演员策略的动作价值，而 Actor 则通过 Critic 提供的价值估计来更新策略，从而实现策略的改进。

具体而言，Actor 网络用于估计在给定状态 s 下采取动作 a 的概率分布 $\pi_\theta(a|s)$ ，其目标是最大化期望回报。

$$\pi_\theta(a|s) \approx P(A_t = a | S_t = s) \quad (1)$$

而对于 Critic 网络，在 PPO 中通常估计状态价值函数 $V_\Phi(s)$ ，利用 $V_\Phi(s)$ 可以构建优势函数 $A(s, a)$ 来指导 Actor 更新：

$$V_\Phi(s) \approx \mathbb{E}[R_t + \gamma V(S_{t+1}) | S_t = s] \quad (2)$$

2.2 算法核心优化

2.2.1 Clip 截断式优化目标

为了避免策略更新步长过大导致性能急剧下降，PPO 引入了 Clip 机制。首先定义概率比率 $r_t(\theta)$ ：

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (3)$$

其中 π_θ 是新策略， $\pi_{\theta_{\text{old}}}$ 是旧策略。

截断目标函数 $L^{CLIP}(\theta)$ 如下所示，旨在为 Conservative Policy Iteration (CPI) 构建下界：

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t) \right] \quad (4)$$

其中 $\hat{A}_t$ 是优势函数的估计值， ε 是一个超参数，通常设置为 0.2，用于限制 $r_t(\theta)$ 在 $[1 - \varepsilon, 1 + \varepsilon]$ 区间内。

- 当 $\hat{A}_t > 0$ 时，目标是增大 $r_t(\theta)$ ，但截断机制限制其最大不超过 $1 + \varepsilon$ 。
- 当 $\hat{A}_t < 0$ 时，目标是减小 $r_t(\theta)$ ，但截断机制限制其最小不低于 $1 - \varepsilon$ 。

这种机制确保了策略更新不会偏离旧策略太远，从而保证了训练的稳定性的。

2.2.2 GAE 广义优势估计

广义优势估计 (Generalized Advantage Estimation, GAE) 用于计算优势函数 $\hat{A}_t$ ，定义为多步时间差分误差的加权和：

$$\hat{A}_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k} \quad (5)$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 是单步时间差分误差。参数 $\lambda \in [0, 1]$ 控制方差与偏差的权衡。

2.2.3 自适应 KL 惩罚

除了截断约束外，PPO 还可以通过在目标函数中加入 KL 散度惩罚项来限制策略更新幅度。带 KL 惩罚的目标函数为：

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta)\hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t)|\pi_{\theta}(\cdot|s_t)] \right] \quad (6)$$

其中 β 是自适应调整的惩罚系数。在每次更新后，根据计算出的 KL 散度平均值 d 动态调整 β ：

- 如果 $d < d_{\text{targ}}/1.5$ ，说明更新幅度过小，减小惩罚： $\beta \leftarrow \beta/2$
- 如果 $d > d_{\text{targ}} \times 1.5$ ，说明更新幅度过大，增大惩罚： $\beta \leftarrow \beta \times 2$

3 实现细节

3.1 工具函数实现

`compute_advantage` 函数用于计算 GAE 优势函数，在接收 TD 误差 δ_t 序列，通过逆序递归方式计算优势函数 $\hat{A}_t$ 。

```
1 def compute_advantage(gamma, lmbda, td_delta):
2     td_delta = td_delta.detach().numpy()
3     advantage_list = []
4     advantage = 0.0
5     for delta in td_delta[::-1]: # 逆序遍历计算 GAE
```

```

6         advantage = gamma * lambda * advantage + delta
7         advantage_list.append(advantage)
8     advantage_list.reverse() # 计算完成后翻转回正序
9     return torch.tensor(np.array(advantage_list), dtype=torch.float)

```

`train_on_policy_agent` 函数则实现了 PPO 训练循环，在一个 Episode 中收集完整的轨迹数据（包括 State, Action, Reward 等），然后用于更新策略的训练。

```

1 def train_on_policy_agent(env, agent, num_episodes):
2     # .....
3     for i_episode in range(num_episodes):
4         transition_dict = {'states': [], 'actions': [], 'next_states': [], 'rewards': [], '
5                             'dones': []}
6         state, info = env.reset()
7         done = False
8         while not done:
9             action = agent.take_action(state)
10            next_state, reward, terminated, truncated, _ = env.step(action)
11            done = terminated or truncated
12            transition_dict['states'].append(state)
13            transition_dict['actions'].append(action)
14            transition_dict['next_states'].append(next_state)
15            transition_dict['rewards'].append(reward)
16            transition_dict['dones'].append(done)
17            state = next_state
18            agent.update(transition_dict)
19        # .....

```

3.2 Actor-Critic 网络设计

这里我构建了两个 MLP 神经网络分别作为 Actor 和 Critic。PolicyNet，也就是 Actor 以状态维度的向量作为输入，经过一个包含 128 维的隐藏层并以 ReLU 为激活函数，最后通过 Softmax 层输出概率分布。ValueNet，也就是 Critic 采用类似的结构，但输出层为一个线性单元，用于输出状态价值的估计。

```

1 class PolicyNet(torch.nn.Module):
2     def __init__(self, state_dim, hidden_dim, action_dim):
3         super(PolicyNet, self).__init__()
4         self.fc1 = torch.nn.Linear(state_dim, hidden_dim)
5         self.fc2 = torch.nn.Linear(hidden_dim, action_dim)
6
7     def forward(self, x):
8         x = F.relu(self.fc1(x))
9         return F.softmax(self.fc2(x), dim=1)

```

```

11 class ValueNet(torch.nn.Module):
12     def __init__(self, state_dim, hidden_dim):
13         super(ValueNet, self).__init__()
14         self.fc1 = torch.nn.Linear(state_dim, hidden_dim)
15         self.fc2 = torch.nn.Linear(hidden_dim, 1)
16
17     def forward(self, x):
18         x = F.relu(self.fc1(x))
19         return self.fc2(x)

```

3.3 策略更新逻辑

在策略更新阶段，计算 Actor 的损失时，这里采用了 Clip 截断机制与自适应 KL 惩罚相结合的策略：一方面通过 `torch.clamp` 来限制概率比率的变化范围，另一方面计算新旧策略的 KL 散度作为惩罚项加入总损失中。此外，在每次更新结束后，还会根据 KL 散度的实际值动态调整惩罚系数 β ：

```

1 def update(self, transition_dict):
2     # .....
3     td_target = rewards + self.gamma * self.critic(next_states) * (1 - dones) # 计算时间差分
        目标
4     td_delta = td_target - self.critic(states) # 计算时间差分误差
5     advantage = utils.compute_advantage(self.gamma, self.lmbda, td_delta.cpu()).to(self.
        device)
6     advantage = (advantage - advantage.mean()) / (advantage.std() + 1e-8) # 优势归一化
7     old_log_probs = torch.log(self.actor(states).gather(1, actions)).detach() # 旧策略的 log
        概率
8     for _ in range(self.epochs):
9         log_probs = torch.log(self.actor(states).gather(1, actions)) # 新策略的 log 概率
10        ratio = torch.exp(log_probs - old_log_probs)
11        surr1 = ratio * advantage
12        surr2 = torch.clamp(ratio, 1 - self.eps, 1 + self.eps) * advantage # Clip 部分
13        clip_loss = -torch.min(surr1, surr2)
14        kl_div = torch.mean(old_log_probs - log_probs)
15        kl_loss = self.kl_beta * kl_div # KL 惩罚项
16        actor_loss = torch.mean(clip_loss + kl_loss)
17        critic_loss = torch.mean(F.mse_loss(self.critic(states), td_target.detach()))
18        self.actor_optimizer.zero_grad()
19        self.critic_optimizer.zero_grad()
20        actor_loss.backward()
21        critic_loss.backward()
22    # .....
23    with torch.no_grad():
24        kl_d = torch.mean(old_log_probs - new_log_probs).item() # 计算所有 epoch 训练完成之后最
        新的 KL 散度

```

```

25     if kl_d < self.kl_target / 1.5:
26         self.kl_beta /= 2
27     elif kl_d > self.kl_target * 1.5:
28         self.kl_beta *= 2

```

3.4 超参数设置

训练中具体的超参数设置如下表所示，另外优化器选择了 AdamW 优化器。

表 1: PPO 算法超参数设置

参数名称	变量名	设定值
Actor Learning Rate	<code>actor_lr</code>	1e-3
Critic Learning Rate	<code>critic_lr</code>	1e-2
Hidden Layer Size	<code>hidden_dim</code>	128
Discount Factor	<code>gamma</code>	0.98
GAE Parameter	<code>lmbda</code>	0.95
Clip Epsilon	<code>eps</code>	0.2
KL Target	<code>kl_target</code>	0.01
Initial KL Beta	<code>kl_beta</code>	0.5
Number of Episodes	<code>num_episodes</code>	500
Update Epochs	<code>epochs</code>	10

4 实验结果与分析

4.1 训练过程分析

如图 1 所示，经过 500 个 episode 的训练，平均 returns 逐渐提升，并最终稳定在接近 500 的水平，可以认为模型已经学会了保持倒立摆的平衡。可以看到实际上在约 150 个 episode 的时候模型的平均 returns 已经有了显著的提升并接近 500，但是此后的训练过程中出现了较大的波动。该图已经经过平滑处理，在原始曲线图像中波动幅度更大，虽然整体趋势保持了一致，但这仍然反映出强化学习训练过程中的不稳定性。

图 2 和图 3 分别展示了训练过程中 Actor 和 Critic 的损失值变化情况。可以看到，随着训练的进行，Actor 损失值由于其定义，是从负值开始逐渐趋近于零，虽然其波动非常大，但是仍然能看出相比训练初期有所改善，更加趋近于 0。另一边，Critic 损失值则没有表现出明显的趋势，除部分 episode 出现较大的波动外，整体维持在一个相对较小的正值，这与每一回合的奖励和状态转移都带有随机性有关，因此 Critic 损失值收敛到 0 是合理的。

图 4 展示了训练过程中概率比率的变化情况。可以看到，随着训练的进行，ratio 值在 1 左右叫小范围波动，这表明新策略与旧策略之间的差异在逐渐减小。同时也可以注意到受算法中 Clip 截断以及自适应 KL 惩罚的影响，ratio 值并未出现过大的偏离，这验证了这些方法子啊限制策略更新幅度方面的有效性。

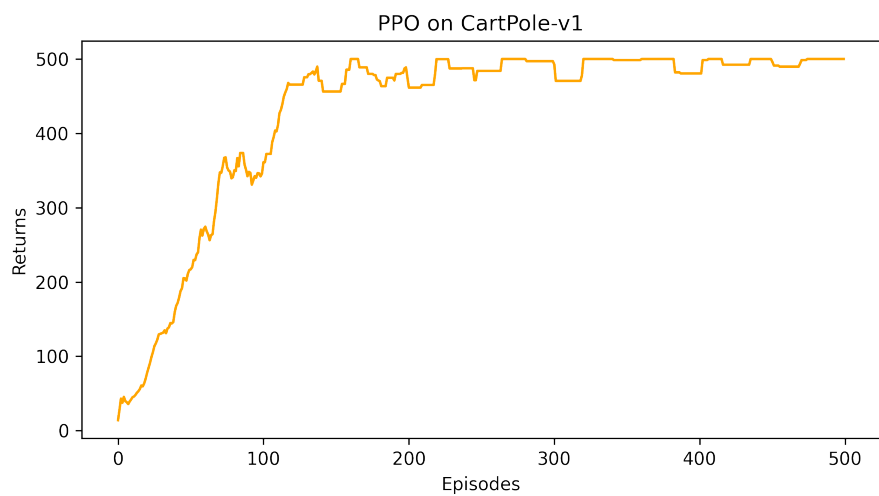


图 1: 训练过程中 returns 的变化趋势



图 2: 训练过程中 Actor 损失值的变化

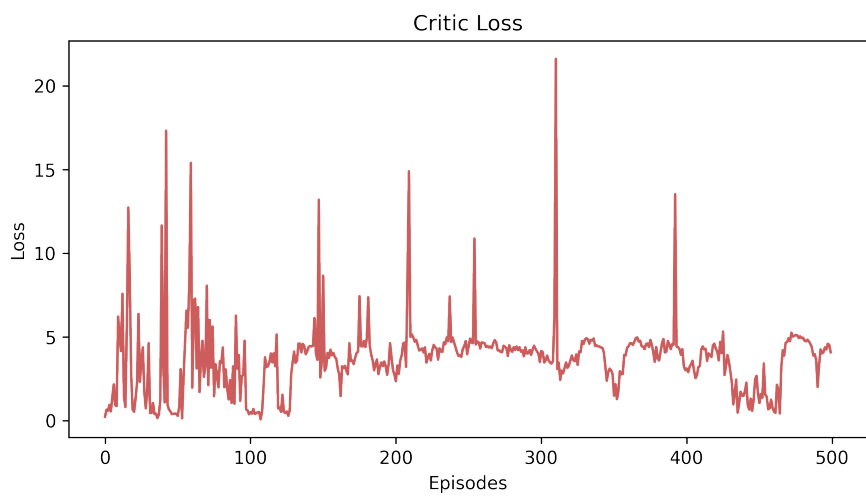


图 3: 训练过程中 Critic 损失值的变化

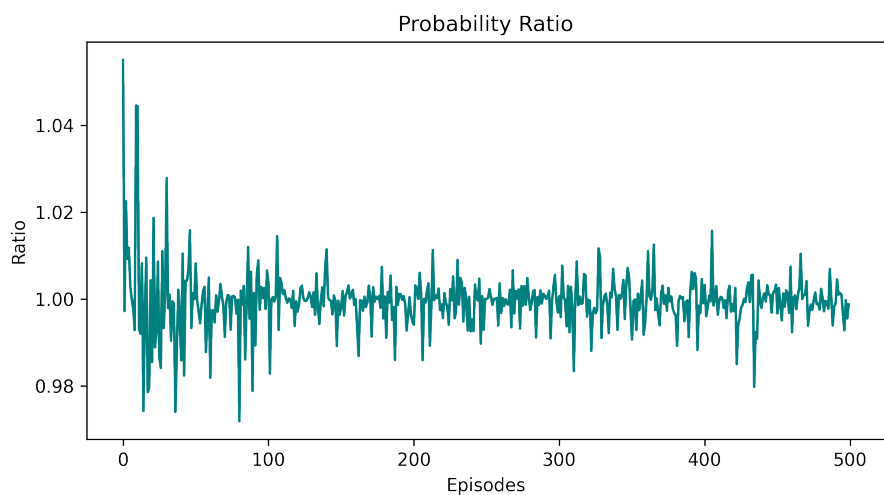


图 4: 训练过程中 ratio 的变化

4.2 模型演示情况

经过 500 个 episode 的训练，在实际演示过程中，模型在共计 5 次尝试中均成功将倒立摆保持在平衡状态达到了最大步数 500 步。

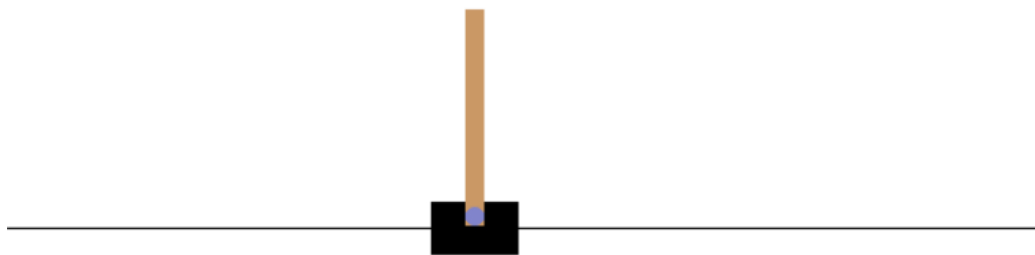


图 5: 实际运行情况

5 总结

本次作业中，成功实现了 PPO 算法并应用于倒立摆控制任务。通过 Clip 截断机制和自适应 KL 惩罚，有效提升了训练的稳定性和策略的性能。最终的模型能够稳定实现倒立摆 500 步的控制目标。